ulang: An Agent-First Programming Language

Design Rationale, Current State, and the Road Ahead

Gidon Ernst

November 2026 — internal design paper, not versioned

Abstract

Writing code has become nearly free; reading, checking, and changing it safely has not. When the main author of a program is an AI agent, the agent's typing does not cost much. What costs is two other things: the agent learning the language from the few pages in its context, and someone (a person, a tool, or another agent) verifying what it wrote. This paper describes *ulang*, a small language designed under that assumption, and implemented in itself: a parser, a type checker with Hindley–Milner inference and checked exceptions, an interpreter, a compiler to C, and a prover for lemmas about pure functions, about 19 000 lines of ulang that compile themselves to a fixed point. We state the design principles that follow from the agent-first assumption: locality of every rule, one way to write each construct, a canonical layout enforced by the checker, effects visible in signatures and at calls, atomic effects under exceptions, and failure that is never silent. We then describe the language as it will stand once the plans now in progress are in. These are diagnostics as a machine-readable protocol with hints for the habits of other languages, examples that are part of signatures and run as tests, lemmas that are tested on enumerated values before they are proven, a structured proof language over an e-graph simplifier, lint rules that are equalities the prover establishes, and separate compilation. For each property that makes code cheap to verify, we say how the design provides it and where it falls short. We close with the tensions that remain and the experiments that would settle them.

Contents

1 Introduction **2**

2 Design Principles **3**

3 The Language **5**
- 3.1 A first example . . . 5
- 3.2 Functions, procedures, and modes . . . 6
- 3.3 Exceptions, checked and inferred . . . 6
- 3.4 Datatypes, fields, and pattern aliases . . . 6
- 3.5 Containers as forms . . . 7
- 3.6 Overloading without preferences . . . 7
- 3.7 Modules and packages . . . 7
- 3.8 Tests as part of the language . . . 8

4 Feedback as a Protocol **8**
- 4.1 Positions on every part . . . 8
- 4.2 All errors in one run . . . 8
- 4.3 Hints for the habits of other languages . . . 8
- 4.4 Seeing what the checker resolved . . . 9
- 4.5 Refactorings that are safe under overloading . . . 9

5 A Ladder of Evidence **9**
5.1 Examples in signatures . . . 9
5.2 Lemmas, tested before they are proven . . . 10
5.3 The default proof and structured proofs . . . 11
5.4 Contracts . . . 11

6 The Prover **12**

7 Lint as Proven Rewrites **13**

8 Implementation **14**
8.1 Phases as packages . . . 14
8.2 Bootstrap . . . 14
8.3 Backends . . . 14
8.4 Speed of the loop . . . 15
8.5 Documentation that is checked . . . 15

9 Status **15**

10 Reconsidering the Design **15**
10.1 What the design gets right . . . 15
10.2 Tensions that remain . . . 16
10.3 What we would add, beyond the plans . . . 17

11 Related Work **17**

12 Conclusion **18**

1 Introduction

Most conventions of programming languages grew around human authors. Short notation saves typing, implicit conversions save thought, and the many ways to write a loop let each author keep a personal style. Readers pay for all of this, in context they must recover and in alternatives they must recognise. While writing was the expensive part, the trade was reasonable.

Coding agents change both sides of that trade. An agent writes thousands of lines without fatigue, so a keyword more or less costs it nothing. On the other side, agents make a kind of mistake that conventional languages hide well: well-formed, plausible code that misreads the intent. A shadowed name, an implicit fallback, a forgotten case, an effect that happens somewhere it should not. Whoever checks the code (a person, a reviewing model, a tool) now dominates the cost. Following the companion sketch on non-functional properties of AI code, we take *cost to verify* as the measure of structural quality: how much a verifier must know and check to be confident that a unit is correct and that a change to it is safe.

A new language meets a second problem, one particular to agents: there is no ulang in any model's training data. An agent that writes ulang learns it inside one session, from a page of documentation and from the messages of the checker. Two things follow.

- **The cost of a feature is the length of the paragraph that explains it to an agent.** A feature that needs a long explanation is expensive however much it saves in typing.
- **The checker is the teacher.** Every error message is a lesson. An error that names only the token expected (`end expected but x found`) teaches little. An error that names the construct to write instead teaches the language.

ulang is an experiment in taking these premises seriously. It has pure functions and imperative procedures, algebraic datatypes, Hindley–Milner inference [14] within declarations, checked exceptions, unbounded integers, ordered containers, comprehensions, modules with explicit exports, and lemmas with an automatic prover. It is implemented in itself, and the implementation has been developed largely with coding agents, so the language is also its own main test of whether it suits them. A file of trip-ups, `unclear.md`, records each place where an agent guessed wrong, so that the documentation can be fixed from it.

Scope of this paper. Several plans are in progress as notes in the repository: diagnostics (`diagnostics.md`), examples and tested lemmas (`examples.md`), lint (`lint.md`), the prover and its proof language (`prove.md`, `prover.md`), separate compilation (`archives.md`), and a native backend (`assembly.md`). We describe the language *as if those plans were done*, since that is the design to discuss. Where something is not yet implemented we mark it with a superscript ^{P}, and Section 9 gives the status of each part. Items of `TODO.md` that no plan covers yet are discussed as open questions in Section 10.

Contributions.

1. A set of design principles for a language whose authors are agents, derived from cost to verify and in-context learnability (Section 2).
2. A language that applies them throughout, and a self-hosting implementation as evidence that the result is practical (Sections 3 and 8).
3. A view of diagnostics as a protocol between checker and agent rather than text for people (Section 4).
4. A *ladder of evidence* that runs from types through examples, enumerated tests, and the default proof up to structured proofs and contracts, where each rung reuses the one below (Section 5).
5. Lint as extraction from an e-graph: a suggested rewrite is a cheaper representative of a class that the prover's rules populate, so every suggestion is a proven equality (Section 7).
6. An account of what remains in tension (Section 10).

2 Design Principles

The principles below are those of `DESIGN.md`, sharpened by the agent-first premise. Each one names the property of cost to verify that it serves; the properties are those of the sketch on non-functional properties (locality, explicitness, non-redundancy, proportionality, fail-loud behaviour, grounding, traceability, uniformity, and proportionate defensiveness).

P1. Locality.
Every rule can be decided from the code at the point of use, its arguments, and the signatures it refers to. It never depends on other uses in the same command, on line breaks, or on preferences among candidates. Overloading resolves uniquely or fails (Section 3.6). Imports are not transitive. User-defined notations are the one deliberate exception. *Serves: locality, explicitness.*

P2. One way to write each thing.
Two syntaxes for one construct are two things to learn and a chance to mix them up. Lists are `List(1, 2)`, never `[1, 2]`. A chain of conditions is a nested `if`, never `elif`. Rejected alternatives are recorded with their reasons, so that no later session adds them back. *Serves: uniformity.*

P3. Keystrokes are cheap; tokens read are not.
A convenience that only saves typing is not worth a rule. Every statement starts with its keyword (`local`, `assign`, `call`, `raise`, `skip`), and every argument of a call carries its mode (`in`, `out`, `ref`). This verbosity is low in entropy: a reader skims the keywords and gains certainty about the shape. Semantic redundancy, such as restating in a comment what a name says, is what costs context, and the style rules forbid it. *Serves: explicitness.*

P4. The layout is canonical, and the checker enforces it.
`ulang format` writes the only accepted layout, and `check` rejects any other. An agent cannot introduce style drift, and a diff never reformats code it does not change. The formatter is also a test: every file that checks has passed a round trip through parser and printer. *Serves: proportionality, uniformity.*

P5. Comments are syntax.
Prose is a *doc* attached to a module, a command, a constructor, or a `case`. It is kept in the tree and printed back, so no tool loses or moves it. A comment at the end of a line or in the middle of a body is a parse error: the code there needs a name instead. Docs that make claims become *examples*, which are checked (Section 5.1). *Serves: traceability, grounding.*

P6. Effects show in signatures and at calls.
The kind of a declaration is a one-bit effect system: functions are pure expressions, procedures are statements. Data flow shows in the modes of formals and arguments. What a procedure may raise is part of its type. *Serves: locality, explicitness.*

P7. Effects are atomic.
A procedure works on copies of its outs and refs, which reach the caller only when it returns. The handlers of a `try` see the variables as they were before it. There is no state that is half updated when an exception escapes. *Serves: locality, fail-loud.*

P8. Failure is never silent.
Functions cannot `raise`; they fail only by the exceptions of builtins, which are defects. A failed `assert` cannot be caught. Matches are exhaustive. Lookups give options, and a default is named at the call (`value_or`). Generators do not skip elements whose shape does not match. A handler of an exception that its body cannot raise is an error. *Serves: fail-loud, proportionate defensiveness.*

P9. Determinism.
Integers are unbounded. Maps and sets are ordered by a total order on all values, so iteration never depends on hashing. The interpreter and compiled code agree, with the texts of shared runtime errors in one place. The prover works within named budgets and reports when one runs out. Tests of properties enumerate small values rather than drawing random ones. *Serves: grounding, reproducibility of every result an agent sees.*

P10. A small type system, with contracts for the rest.
Types are the decidable, always-checked part of a specification: inference inside bodies, declared types at boundaries, no higher kinds, no computation on types. Everything more expressive goes into examples, lemmas, and contracts, which are checked by tests or the prover (Section 5).

P11. Every departure from a common habit has a hint.
This principle is new to this paper. Agents arrive with the habits of Python, Rust, and TypeScript, and ulang rejects some of them on purpose. A rejection is a learning cost, which a hint that names the replacement pays back (Section 4.3). A departure that cannot be given such a hint is a sign that it is arbitrary.

```
data Account :=
  case Account(owner: Str, balance: Int)

data Overdrawn :=
  case Overdrawn(missing: Int)

// takes amount from acct, which stays as it was if the balance does not cover it
procedure withdraw(in amount: Int, ref acct: Account) raises Overdrawn :=
  if acct.balance < amount then
    raise Overdrawn(amount - acct.balance)
  end
  assign acct := acct with balance := acct.balance - amount

procedure transfer(
  in  amount: Int,
  ref from: Account,
      to: Account
) raises Overdrawn :=
  call   withdraw(in amount, ref from)
  assign to := to with balance := to.balance + amount

function totals(accts: List[Account]): Map[Str, Int] :=
  Map(for a in accts yield a.owner => a.balance)

lemma size_append[A](xs: List[A], ys: List[A]) :=
  ensures size(xs ++ ys) == size(xs) + size(ys)

test kept() :=
  local a := Account("ada", 10), b := Account("bob", 0)

  try
    call transfer(in 15, ref a, b)
  catch
    case Overdrawn(_) =>
      skip
  end
  assert a.balance == 10 and b.balance == 0
```

Figure 1: A program in ulang. `ulang test` prints `test kept passed` and `lemma size_append proved`.

Two consequences shape the rest of the design. First, *the parser desugars nothing.* Comprehensions, `for` loops, patterns in lambdas, field updates, and pattern aliases stay as written until the checker translates them, so the formatter prints what the source wrote and an error is reported against what the agent wrote. Second, *tools are part of the language.* Formatting, resolution of names, refactorings, lint, and the prover read and write the same syntax tree, and the commands of one binary are the interface through which an agent works.

3 The Language

3.1 A first example

Figure 1 shows a complete program. It checks, its tests pass, the lemma is proved automatically, and its layout is the one `format` writes.

The example shows most of the core. Datatypes declare every constructor after `case`, and

every field has a name. Fields are read as `e.f` and updated as `e with f := v`. Procedures take formals grouped by mode, and calls repeat the modes. Exceptions are values of datatypes, and a procedure says what it raises. The handler in the test sees `a` and `b` as they were before the `try`, so the final assertion holds even though `withdraw` was reached. A comprehension builds a map, and a lemma states a property of a library function. The doc above `withdraw` is part of the syntax.

3.2 Functions, procedures, and modes

Functions are expressions: `let p := a, q := b in e` binds patterns in parallel, `if c then a else b` always has its `else`, and `match` is exhaustive and checked for unreachable cases. Procedures are statements: `local`, `assign` (simultaneous: `assign a := b, b := a` swaps), `call`, `if`/`while`/`for` with `end`, `try`/`catch`, `raise`, `assert`, and `skip`. There is no `return` and no `break`. A procedure gives its results by its outs, and a loop ends by its condition or an exception. Variables are read only where they are assigned on every path, and outs must be assigned on every path that does not raise.

Both kinds are values. `lambda x, (a, b) => e` is a function whose formals are irrefutable patterns. `lambda in x, out y => ... end` is a procedure of type `(in A, out B) raises E`, which sees the variables around it by value and assigns only its own outs and refs. Procedure values abstract over effectful code: the parser of ulang writes its separated lists once, as a procedure that takes the parser of an element.

No formal, local, or pattern variable may hide a local around it (`HiddenLocal`). Shadowing is a classic source of plausible-looking bugs, and an agent rarely means it.

3.3 Exceptions, checked and inferred

A raised value is of a datatype without type parameters. `raise` is a statement, so only procedures raise. What a procedure may raise is checked after its package:

- A statement raises the types of its `raise`, the `raises` of the procedures it calls (also through variables of procedure types), and what a `try` lets through: the types that its unguarded handlers do not cover, decided by the exhaustiveness check.
- Exported procedures and those with `raises` raise nothing they omit. Private procedures without `raises` infer theirs, as the least fixpoint over all of them. Annotation is required only at boundaries, where a reader needs it.
- Procedure types carry their raised types as a set, compared exactly, without subtyping. A lambda gets an unknown that its place of use fixes.
- A handler of a type that its body cannot raise is an error. So is a handler that nothing in its `try` reaches. Dead defensive code is not allowed to accumulate.

The exceptions of builtins (`DivisionByZero`, `OutOfBounds`, `NoCharacter`, `MissingField`) are not counted, since functions cannot declare them. They are defects of the program, and the planned `requires` of functions rules them out statically (Section 5.4).

3.4 Datatypes, fields, and pattern aliases

Datatypes are sums of records. A field name may repeat across constructors with the same type, and an accessor applied to a constructor without the field raises `MissingField`. *Pattern aliases* name a shape at the level where code works with it:

```
pattern ECall(name: Name, typ: Type, args: List[At[Expr]]): Expr :=
  EApp(AOk(_, EFun(name, typ)), args)
```

An alias is typed once in its module and expanded at its uses, so exhaustiveness and the backends see only constructors. An alias without _ also builds values. Unlike view patterns, aliases call no function, so exhaustiveness stays decidable and matching always terminates.

3.5 Containers as forms

The containers are written as *forms* named like their types: `List(a, b; rest)`, `Set(a, b)`, `Map(k => v)`, `Array(a, b)`. Comprehensions use the same form with generators and guards from left to right:

```
List(for x in xs for y in ys if x < y yield (x, y))
Map(for k => v in m if v > 0)
```

The form names what is built, so the result never depends on the expected type. That is the problem of Scala's `CanBuildFrom`, and it would violate locality. The checker translates forms into the folds of the container's module, chosen by the container's type. Generators range over lists, sets, maps, arrays, buffers, and options alike, and `for x in c if q do ... end` is the matching statement. Their patterns are irrefutable: a refutable pattern would either drop elements silently or fail at run time, and both were rejected. The shared names (`size`, `contains`, `get`, `remove`, `foldl`, `foldr`, `++`, `--`) take their arguments in the same order for every container, so knowing one container means knowing them all.

3.6 Overloading without preferences

Within a module, a name overloads only by arity. Across modules it overloads by type. A use infers its arguments, keeps the candidates of its arity whose instance unifies with them and with the expected type, and commits if exactly one remains. Pending uses are solved to a fixpoint at the end of the command. Since the substitution only grows, a candidate that does not fit never fits again, so the result does not depend on the order of solving. There are no preferences, no defaulting, and no implicit conversions. An ambiguous use is an error that lists the candidates with their types:

```
e3.u: type error: empty is ambiguous: prelude/set.empty: () -> Set[A],
  prelude/map.empty: () -> Map[K, V], prelude/buffer.empty: () -> Buffer[A] at 2.14
```

The planned hint turns this into the annotation that decides it, `local s: Set[Int] := empty()`[P].

The *hooks* `compare`, `==`, and `show` follow the same rule. A function of the right shape in the module of a datatype becomes that type's order, equality, or text, and the builtins reach it through the type descriptor of each value, also inside other values and in generic code. Only abstract types may declare `==`. Comparing values of an abstract type without an order raises `Unordered`, since the shape of an AVL tree is no order.

3.7 Modules and packages

A file with `export` and `implementation` sections is a module, and the modules of a directory form a package. Members see each other's exports, also cyclically. Packages are imported by their path from the root of the project, `import "ulang/syntax"`, and imports are not transitive, so the import lines of a file name every package its names come from. Export sections hold signatures without bodies, which the implementation repeats exactly, so an agent that reads an export section sees the whole interface. Abstract types hide their constructors. Relative imports, re-exports, and subpackages private to a parent were each considered and rejected: each would add a second way to reach the same declaration, or a second spelling of the same package.

3.8 Tests as part of the language

Tests are commands, `test name() := ...`, next to the code they test, where they see private declarations. `assert raises p do ... end` expects an exception that matches `p`. Errors that a phase must report are themselves tests: a case is a source text in a test of `ulang/test/errors/`, run through the phases, with the expected exception matched by `assert raises`, including its position. Every file of the repository passes. There are no "expected to fail" files, so "all files pass" is the one rule to check.

4 Feedback as a Protocol

An agent works in a loop: write, check, read the errors, fix. The quality of that loop depends on three things: how many errors one run reports, how precisely each is placed, and whether it says what to do.

4.1 Positions on every part

Every part of an expression, statement, and pattern is a value of

```
data At[A] :=
  case AOk(pos: Pos, that: A)
  case AErr(pos: Pos, that: A, error: Diagnostic)
  case AUnparsed(pos: Pos, text: Str, error: Diagnostic)
```

Positions are never stripped. They reach the checked program, the runtime errors[P], and the open goals of the prover[P]. Code that does not care reads `x.that`. A type error is reported at the innermost part around it, and a part of the wrong type is a `Mismatch` at that part, with the type expected, the type found, and the role of the part:

```
e2.u: type error: expected Int but found Str in argument 1 of the call at 2.3
e2.u: type error: match is not exhaustive, missing None at 5.3
```

4.2 All errors in one run

The checker records each error and continues with a fallback: a fresh type for an expression, a statement counted as assigning what it assigns, a pattern as a wildcard. So one run reports the errors of all commands of the first package that has some, at most 20 per file, without cascades from the fallbacks. The parser resumes at the next command, and in the planned recovery also at the next statement and argument[P]. What does not parse becomes an `AUnparsed` part, and the commands that parse are checked.

Errors are data from start to end. Each is a `Diagnostic(kind, code, module, pos, message, details)`, where `code` is the constructor of the error and `details` holds pairs such as `expected`, `found`, `candidates`, and `missing`. The driver prints them as one JSON object per line[P], the only output format, and the exit status separates source errors, usage errors, failed tests, and internal errors[P]. The editor extension, scripts, and agents all read the same records.

4.3 Hints for the habits of other languages

The checker and the parser often know the fix. The most frequent mistakes of agents writing ulang are habits from other languages, and today they get only the token that was expected:

```
e1.u: parse error: declaration expected but x found at 5.3     -- from:  x := 1
```

The plan attaches a hint to each such habit[P] (Table 1). Each hint has a test case. The `end` of an `if` after `else` is reported at the `if` it closes, found by indentation, instead of at the end

Habit	Hint names
`x := e` as a statement	`assign x := e`
`f(x)` as a statement	`call f(in x)`
`return e`	a function is its expression; a procedure assigns its outs
`[1, 2]`, `x :: xs`	`List(1, 2)`, `List(x; xs)`
`elif`	`else if c then ... end`, with its own `end`
`y += 1`	`assign y := y + 1`
`s + "x"` on strings	`++`
procedure used as a function	`call f(in ..., out r)`
`local` after a statement	locals are declared at the start of a body
unknown name	the nearest visible names, and packages not imported that export it
argument order	a permutation of the arguments that fits a candidate

Table 1: Planned hints for habits that agents bring from other languages. The current messages for the first two are `declaration expected`; for the string case, a cascade of three mismatches.

of the input. Suggested edits, such as the missing cases of a `match` or the annotation that decides an ambiguous use, are listed with the diagnostic and applied by `--apply` as refactorings (Section 4.5).

4.4 Seeing what the checker resolved

Resolution by type is invisible in the text: `size` in a file that imports `set` and `map` may be any of three. The checked program keeps a position for every part, so a use can be looked up by its position. The tool offers `resolve <file> <L.C>` (qualified name, type, and declaration)[P], `refs` (all uses)[P], and `sig <module>` (the export section with its examples)[P], each also as hover in the editor. These cost almost nothing to build, and they turn "read the imports and guess" into one lookup.

4.5 Refactorings that are safe under overloading

Refactorings are changes of the parsed tree, printed by the formatter, which keeps docs since they are syntax[P]. A refactoring is refused unless the file is formatted, the result is checked again, and it is refused if any other use resolves differently than before. That last check makes renaming safe under overloading without any reasoning about overloading. The planned order is rename, filling in the missing cases of a match, adding a field with its constructions and patterns, and changing a signature with its calls. Extracting and inlining are left out, since textual edits do those well enough. Typed edits are also the strongest lever for proportionality: a change described as "add a field" touches exactly what that change needs.

5 A Ladder of Evidence

The common mistakes of agents are logic errors, not type errors. ulang therefore puts specification in the language, as a sequence of rungs. Each rung costs a little more to write than the one below, gives stronger evidence, and reuses the machinery below it (Table 2).

5.1 Examples in signatures

A signature gives types. A doc gives prose that nothing checks and that goes stale. Tests sit elsewhere, private to their module. An agent that reads an export section therefore never sees a concrete call with its result. Examples close this gap[P]:

Rung	Written as	Checked by
types	signatures	the checker, always
examples[P]	`example gcd(12, 18) == 6`	run as tests, `gcd.example.1`
tests	`test t() := assert ...`	interpreter and compiled code
enumerated lemmas[P]	`lemma l(xs: List[A]) := ensures ...`	all small values, `l.lemma`
default proof	the same lemma	simplification, else induction
structured proof[P]	`proof induction xs case ... end`	the simplifier, step by step
contracts[P]	`requires`, `ensures`, `correct p`	proof obligations of calls and loops

Table 2: The ladder of evidence. A status per obligation (proven, tested, or open) and `ulang audit`, which lists what depends on an assumption, make the rung of every claim visible[P].

```
export

// the greatest common divisor, nonnegative
function gcd(a: Int, b: Int): Int
  example gcd(12, 18) == 6
  example gcd(0, -5) == 5

procedure divmod(in a: Int, b: Int, out q: Int, r: Int)
  example divmod(in 7, 2, out 3, 1)
  example divmod(in 7, 0, out _, _) raises DivisionByZero
```

An example is a closed expression that must apply its own declaration, as overloading resolves it. So `example size(List(1)) == 1` on `set.size` is an error, `ForeignExample`. Examples in an export section see only what an importer sees, so the example of an abstract type builds its values through the exported functions. The checker translates examples into tests named `gcd.example.N`, so runner, interpreter, compiler, and runtime need no change. Examples answer the most common question about a signature whose types leave it open: the order of arguments of `slice`, `split`, or `foldr`. They are the documentation that cannot go stale, and the first values that generated tests try.

5.2 Lemmas, tested before they are proven

A lemma states a property for all formals that satisfy its assumptions:

```
lemma take_drop[A](n: Int, xs: List[A]) :=
  ensures take(n, xs) ++ drop(n, xs) == xs
```

Lemmas are checked as `Bool` expressions over their formals, so they carry resolved names like any code. Before a lemma is proven, the checker generates a test `l.lemma` that enumerates all values of the formals up to a depth, in the style of SmallCheck [18][P]. It is deterministic without a seed, the first counterexample is among the smallest so no shrinking is needed, and small cases are where lemmas about lists and integers fail. Type parameters are instantiated to `Int`. A lemma whose assumptions no small case meets fails as `Vacuous`. Enumerators for each datatype are generated as functions that recur on the depth, so they terminate, and they take the enumerators of type parameters as functions. Abstract types of users are not enumerated, since their constructors do not keep their invariants.

The order matters for agents. A false lemma is refuted in milliseconds with a concrete counterexample, before anyone spends effort on a proof. That gives a property the same tight feedback loop as a type error.

5.3 The default proof and structured proofs

`ulang prove` tries each lemma by simplification, and failing that by induction on one of the formals, with the lemmas proven before as facts. When neither closes the goal it reports the simplified goal, which serves as the error message of a proof. The planned proof language[P] adds as little as possible, with the syntax habits of the rest of ulang: every step starts with its keyword, blocks end with `end`, and cases are written as in `match`.

```
lemma reverse_onto_append[A](xs: List[A], ys: List[A]) :=
  ensures reverse_onto(xs, ys) == reverse(xs) ++ ys
proof
  induction xs
    case List(x; rest) =>
      using reverse_onto_append(rest, List(x; ys)), reverse_onto_append(rest, List(x))
  end
```

The steps are `using` (facts, lemmas, instances, or definitions), `have h := e` with an optional `by`, `induction x`, `match`, `if`, and closing by an instance. Simplification runs before each step and at the end of each sequence. Cases that it closes may be omitted, and a written case that it would close is allowed, so proofs survive a stronger simplifier. The induction hypothesis is the lemma itself, by its name, at any arguments, so generalisation comes for free. Open goals are reported at the step that leaves them, so they appear in the editor like type errors.

5.4 Contracts

Functions get `requires`, which states their domain. That includes the builtins: `requires d != 0` of `/` turns `DivisionByZero` from a defect at run time into an obligation at the call. Facts about the results of functions are lemmas. Procedures get `requires` and `ensures`, where `old(x)` is the initial value of a ref, and a lemma `correct p(in n, out s)` claims that the body meets the contract[P]:

```
procedure sum(in n: Int, out s: Int) :=
  requires n >= 0
  ensures  2 * s == n * (n - 1)
  ...

lemma sum_correct(n: Int): (s: Int) :=
  correct sum(in n, out s)
proof
  at while i < n
    invariant 0 <= i and i <= n and 2 * s == i * (i - 1)
```

Calls use contracts, not bodies. Each body is proven assuming all contracts, including its own, which is sound for partial correctness by induction on the depth of calls (Hoare's rule for recursive procedures [9]). Proofs therefore have no order and no cycles. Loops are specified in the proof, not in the code, either by an invariant or by a *loop contract* `requires P ensures Q` that treats the loop as a recursive procedure [6, 20]. The proof refers to points of the code by quoting them (`at while i < n`), matched modulo positions, which reads without tools and survives most edits. Keeping invariants out of the code keeps the code readable for the agent that maintains it. The proof then becomes a separate artefact, as tests already are.

Two properties make contracts suited to agents. First, every contract is an obligation even without a proof: the verifier tries a default proof and reports where it stops, so the workflow is to write the contract, see where it gets stuck, and add a lemma. Second, an unproven contract makes the proofs that rely on it conditional, and the tool says so (`sum proved, assuming q`). Evidence is never overstated.

6 The Prover

There are no external solvers. The prover is about 2 500 lines of ulang in `ulang/prove/`, working on checked programs, so its terms carry the names the checker resolved. Its builtins are pattern aliases over those names.

Sequents. Goals, facts, lemmas, and induction hypotheses are nested sequents $\forall \bar{x}.\ A_1 \wedge \ldots \wedge A_k \Rightarrow (\exists \bar{y}_1.\ C_{11} \wedge \ldots) \vee \ldots$ whose parts are again sequents or atoms. A lemma *is* a sequent: proving it opens it, and using it adds it unopened as an assumption. Normalisation makes only steps that lose nothing (one-point rules, existentials of assumptions to fixed variables), so it never turns a provable goal into an unprovable one.

Terms. Binders are de Bruijn indices and the variables of the context are levels, so a term of an outer sequent is valid unchanged in inner ones. The state stored at a sequent is a persistent value shared by those nested in it, and splits need no undo.

Contextual simplification. Each part of a sequent is replaced by one that is equivalent in its context. Assumptions become state of the decision procedures (ground atoms), propagating clauses, or rewrite rules (oriented equations). Recursive functions unfold where their outermost `match` or `if` is decided. Linear arithmetic is canonical sums with Fourier–Motzkin elimination and integer tightening. Constructors are injective, distinct, and acyclic. Splits on `if` and `match` are kept only if every branch closes, so the printed goal is always a single goal, equivalent to the original.

The e-graph core^{P}. The current core is a substitution with ground completion. It works, but it is fragile in specific ways. Fuel numbers are spread over the code, matching is syntactic, the first matching rule wins, and an equation that the term order cannot orient, such as commutativity, closes only its exact instance. The plan replaces the term layer with a persistent e-graph with bounded equality saturation [22], in the style of CC(X) [3, 15, 19]. Constructors and literals become class analyses, linear arithmetic is canonised over classes with entailed equalities fed back, and facts, lemmas, and the equations of definitions become rules applied by e-matching, all at once per round. Unfolding becomes guarded e-matching: `size(Cons(x, xs))` fires whenever the argument's class contains a `Cons` node, whatever produced that knowledge. The term order stays, but only to choose the representative that is *printed*, so provability no longer depends on orientation while the open goal stays predictable.

Budgets, not fuel. All limits move into one record of named numbers: nodes, rounds, nested conditions, splits. An open goal that ran out of budget says so, e.g. `(budget: nodes)`, so an agent knows whether to add a fact or to raise a limit. Determinism (P9) holds throughout: the same input gives the same goal, with no timeouts.

Decision procedures, by value for ulang's lemmas. The planned order is: evaluating the order on values (`compare` of constructor terms); `compare` as a theory; ground clauses with bounded splitting; `Bool` and other finite types as splits; constructor exhaustiveness (`xs != List()` gives `xs == List(y; ys)`); division and remainder by literals; integer completeness by the Omega test [17]; and ring normalisation. One hazard has to come first: `Set` and `Map` are AVL trees whose shapes differ for equal values, so their constructors must stay uninterpreted for `==` before `compare` is evaluated. Otherwise the prover would refute true equations about sets.

Property	By construction	By a check (planned)
locality	pure functions, no globals, non-transitive imports, atomic effects	unused imports as errors; `uses os`; unused declarations
explicitness	nothing desugared in the parser; `call`, `assign`, modes at calls	`resolve`; agreement of declared hooks
non-redundancy	one library, shared names across containers	duplicate definitions; library code spelled out
proportionality	canonical format; typed refactorings	a diff of declarations modulo equality
fail-loud	no `raise` in functions, uncatchable `assert`, exhaustive matches, options	swallowed handlers; masked domain errors
grounding	every name resolves or errs; types fix most argument orders	hints for near misses; examples
traceability	—	unused locals, constructors; coverage by tests
uniformity	one way of each construct	rewrites to the canonical form
defensiveness	unreachable cases, unreachable handlers	conditions decided by context; redundant guards

Table 3: The properties of cost to verify, and how ulang meets them.

Termination. Unfolding a function that does not terminate can prove false: `f(x) = f(x) + 1`. The checker will accept recursion in functions only if it is structural or decreases a declared measure, and will advise turning the function into a procedure otherwise[P]. Until then lemmas stay private, and the usage message states the risk.

7 Lint as Proven Rewrites

The design already provides much of what makes code cheap to verify. Table 3 lists each property, what the design gives, and what a check still has to find.

The interesting part is how the rewrites are found. A rewrite is an equality plus a choice of the simpler side, and the e-graph of the prover separates the two[P]:

- **Saturation finds the equalities**, over the definitions of the library, the proven lemmas, and the facts of the path (conditions of `if`s, shapes of `case`s).
- **Extraction chooses** the cheapest representative, where the cost for lint is the number of tokens of source.

Each expression of a body enters the e-graph by the prover's translation, and lint keeps a side table from classes to the parts of the source in them. A lint is reported where a class has a printable representative cheaper than the part by a margin. A representative is printable only if it is a source part or the right side of a rule as written. So a hand-written `match` on an option becomes `value_or(o, d)` because the definition of `value_or` put it in the same class, and `List(for x in xs)` becomes `xs` because a lemma of the library says so. Whoever states a lemma in the library has thereby written a lint rule, which a change of the definition breaks by failing the proof.

Three safeguards keep suggestions honest. A replacement is checked again, and dropped if one of its names resolves differently in the module. A representative whose rule would evaluate an argument that the original reached only under a branch is printable only if that argument is a plain term, which preserves strictness and the exceptions of builtins. Saturation runs within the prover's budget. Conditions decided by the path (*this* `if` *is always true here*) and redundant guards come out of the same saturation, extended per context. Statement-level rules, such as a collecting loop that should be a comprehension or a handler that swallows its exception, stay walks of the syntax.

Package	Lines	Contents
ulang/syntax	1 247	syntax, builtins, notation, errors shared by all phases
ulang/format	1 103	the canonical printer
ulang/parse	3 078	lexer, parser with recovery, loading of packages
ulang/check	5 341	signatures, inference, overloading, exhaustiveness, raises, messages
ulang/run	1 560	the interpreter
ulang/compile	2 057	a typed syntax of C and the compiler
ulang/prove	2 527	terms, sequents, simplification, induction
ulang/ulang.u	914	the driver
ulang/prelude	1 522	the library: lists, options, strings, sets, maps, arrays, buffers, os

Table 4: The self-hosting implementation (November 2026). The repository holds about 2 070 named tests, among them 217 error cases, and 130 lemmas.

Lint is opt-in (`check --lint`). Warnings that every run prints are noise, which agents either chase or learn to ignore, and both are worse than not printing them. `--apply` carries a lint out as a refactoring.

8 Implementation

8.1 Phases as packages

The implementation is a package per phase, each importing only those before it, so the imports enforce the layers (Table 4). The driver `ulang/ulang.u` is a program that imports them all and offers the commands `run`, `test`, `check`, `prove`, `compile`, and `format`. The runtime is about 1 600 lines of C over GMP and the Boehm collector.

8.2 Bootstrap

The seed `bootstrap/ulang.c` is the C of the last compiler that compiled itself to the same C. `make` builds the seed and translates the current sources with it, in about 15 seconds from nothing. `make selfcompile` replaces the seed only if the new compiler reaches the fixed point. A rule that the seed's checker would reject in the new sources (such as the exact comparison of raised types) takes two seeds. The way through is a copy of the sources without the new annotations. That procedure is recorded, because it is exactly the kind of step an agent would otherwise rediscover by failing.

8.3 Backends

The interpreter and the compiler share the checked program and the texts of runtime errors. The compiler builds C as a small typed syntax with a printer that adds parentheses. Each datatype gets one type descriptor with its order, equality, and show. Outs and refs are copied in and out, `try` saves the variables its body assigns before `setjmp`, and comprehensions over lists and arrays are fused into loops. Every value is a uniform `u_value`, with no specialisation by type. That costs some speed, and it is what makes separate compilation simple. The plan^{P} compiles each package once into a static archive, the library included, with mangled names that are stable across translation units and a stamp symbol per archive, so that a stale archive fails to link rather than to run. A second backend^{P} writes x86-64 assembly from the same typed C, so that programs need only an assembler: on the compiler itself, `cc -O2` takes 11.2 s where `as` takes 0.2 s, for a measured slowdown of about 1.35× in a simple backend that keeps every variable in memory.

8.4 Speed of the loop

What matters for an agent is how long one round of feedback takes. Checking the whole compiler takes about 0.9 s, and the test suite of the project about 9 s interpreted, with the error cases compiled. The interpreter is about 300 times slower than compiled code. A faster interpreter was rejected; the quick loop on large inputs is compiled tests instead, with one tool command that builds and caches the binary of the tests[P].

8.5 Documentation that is checked

The documentation is what an agent learns the language from, so it is tested like code. `checks/docs.u` verifies that every reference in `CLAUDE.md`, `README.md`, and `TODO.md`, written as a list of names in backticks followed by "of" and a file, names declarations of that file. A rename that forgets the documentation fails the build. The planned reference `LANGUAGE.md`[P] separates what a writer of programs needs (the constructs by example, the semantics of effects, the library, and the differences from other languages) from the implementation notes, which are loaded into every session today and cost context.

9 Status

Implemented	core language, inference, overloading, hooks, forms and comprehensions, pattern aliases, modules and packages, checked and inferred `raises`, atomic effects, canonical format enforced by `check`, docs as syntax, positions on every part, all type errors of a package in one run, parser resumes at the next command, interpreter, compiler to C, compiled test suite, lemmas with the default proof, editor extension
Planned, with a plan	JSON diagnostics, exit statuses, recovery inside commands, hints (`diagnostics.md`); examples and enumerated lemma tests (`examples.md`); e-graph core, budgets, decision procedures, proof language (`prover.md`, `prove.md`); lint by extraction (`lint.md`); archives per package (`archives.md`); assembly backend (`assembly.md`)
Open, in `TODO.md`	`resolve`/`refs`/`sig`; refactorings; tests with formals; `LANGUAGE.md`; exceptions as values; termination check; unused imports as errors; stopping folds and bounded quantifiers; qualified names; `os` as a package; a single build tool; positions in runtime errors; contracts and `correct`; evidence per obligation; `uses os`; replay of `os` results

Table 5: Status of the parts described in this paper.

10 Reconsidering the Design

With the plans assumed done, this section asks where the design is still in tension with its own premises, and what we would change or test.

10.1 What the design gets right

The function/procedure split is the cheapest effect system that pays. One bit per declaration, visible in its keyword, gives most of the value of effect rows: a function cannot read a file, raise, or loop forever (once termination is checked). Checked exceptions with inference for private procedures put the annotation burden only at boundaries, where readers need it. `uses os` on exported procedures[P] would add the remaining capability at the same price.

Atomic effects remove a whole class of review. In most languages, reviewing a handler means asking which assignments in the `try` happened before the exception. In ulang the answer is always "none". The plain reading of Figure 1 is the correct one, and an agent reasoning locally reaches it.

Uniqueness without preference is the right overloading for agents. Preferences ("the most specific", "the nearest") are rules that a reader must simulate. ulang's rule needs no simulation: a use compiles only if exactly one candidate fits, and otherwise the error lists the candidates. The planned `resolve` makes the remaining invisibility a single lookup.

Canonical form and docs as syntax compose. Because the layout is canonical and docs are nodes, every tool that changes the tree (refactorings, `--apply` of lints, examples printed back) can print the whole file without losing anything. This is the precondition for typed edits, and few languages meet it.

10.2 Tensions that remain

Novel syntax against zero training data. ulang deliberately breaks habits (`x := e`, `[1, 2]`, `+` on strings, `return`). Each break is a learning cost for an agent with no ulang in its priors. The bet is that explicit, uniform syntax plus hints (P11) costs less than familiar but ambiguous syntax. We have anecdotes: `unclear.md` records about thirty trip-ups, mostly in the implementation rather than the language, and the habits of Table 1 are those observed. We do not have a measurement. *Proposed experiment:* a fixed set of tasks, given to agents with only `LANGUAGE.md`, measuring the rounds to a passing program with and without hints, and against a variant with familiar spellings for the same semantics. If the familiar variant wins clearly, the forms and keywords are worth revisiting. The semantics, which are where the value lies, would stay.

Locality has three exceptions. User-defined notations make parsing depend on imports. Cross-module overloading makes resolution depend on imports. Hooks make `==` on a type depend on a function in its module. Each was kept for a reason: notations for the prelude's operators, overloading for shared container names, hooks for abstract types. The hooks are the most worrying, since a declared `compare` must agree with structural equality, and nothing checks that today. *Recommendation:* generate the agreement as a lemma, tested on enumerated values and proven where possible, and make the hooks a line of `sig` output, so that they are seen where the type is.

Argument order is the residual grounding hazard. Named fields protect constructors, and types protect most calls. They do not protect `slice(i, j, s)` or `split(sep, s)`. Named arguments at calls would protect them, but they add a second way to write every call, against P2. The design's answer is examples in signatures, plus a hint that tries permutations of the arguments. We think that is right, and suggest making it a style rule with a check: an exported function with two formals of the same type must have an example.

Exceptions cannot become values. Code that collects errors (the driver, the test helpers) catches each kind into an out and lists every phase in long `raises`. This is the most visible awkwardness in the sources. A `try` with a result, or a statement that gives an `Option` of the exception of a call, together with the errors of all phases as one datatype, would fix it without weakening checked exceptions. We would do this before the contracts, since the prover's obligations for `raise` paths become simpler with it.

Partial field accessors. `e.f` raises `MissingField` for a constructor without the field: a partial function in a language that otherwise makes failure visible. Marking partial access or rejecting it statically were both considered and dropped. With `requires` on functions and the checker knowing which constructors have a field, the accessor could carry an implicit `requires e is C1 or ...`, turning the defect into an obligation, consistent with the treatment of `/`.

The library is ambient. Every module sees the whole library, including `os`, without imports. That keeps short programs short, but it makes every library name a candidate everywhere and hides I/O. Moving `os` into its own package, which must be imported, is planned and costs a line per program that does I/O. We consider that line a feature, as a declaration of capability.

Verbosity has a reading cost after all. P3 says keystrokes are cheap. But context windows are finite, and `CLAUDE.md`, which describes the language, its implementation, and the workflow in sentences that carry a reference in every clause, now has about 9 000 words, loaded into every session. The fix is not a terser language. It is separating what writers of programs need (`LANGUAGE.md`) from what implementers need, and measuring the paragraph cost of each feature, as P1–P11 suggest.

Budgets make proofs deterministic, not robust. A proof that closes at one budget may stay open at a smaller one, and a stronger simplifier can only close more goals, never fewer. That holds only if extraction and saturation are monotone in the facts, which an e-graph gives, but a budget of nodes cuts across. Reporting exhaustion is necessary. A test that runs the corpus of lemmas at half the budget and lists the ones that open would show how much headroom proofs have.

Functions without parameters are computed once. They are effectively global constants, initialised lazily. That is safe because functions are pure, but a non-terminating one (until termination is checked) deadlocks its first use. The termination check closes this.

10.3 What we would add, beyond the plans

- **A rejected-habits test suite as a specification of P11**: one case per habit, with the hint expected, so that the documentation and the checker cannot drift apart.
- **A paragraph budget**: each feature's explanation in `LANGUAGE.md` is measured, and a new feature states its paragraph before it is implemented, as the cost P1–P11 accept.
- **Evidence in the output**: `test` and `prove` report, per declaration, the highest rung of Table 2 it reaches, so that an agent (or a reviewer of an agent's change) sees which claims are proven, which are only tested, and which are open.
- **Replay of runs**: `run` records the results of `os` procedures and replays them, so that a failure an agent observed can be reproduced by a human exactly. This follows from P9 and costs little, since all I/O goes through one module.

11 Related Work

Modes and contracts. The `in`/`out`/`ref` modes come from Ada and SPARK [1], whose discipline of data flow ulang shares. Contracts follow Eiffel [12] and the auto-active verifiers Dafny [11] and Why3 [7]. ulang differs in keeping proofs and loop specifications out of the code, in using no external solver, and in putting enumerated tests below proofs on the same statements.

Effects. Koka [10] tracks effects as rows. ulang's function/procedure split with checked, inferred exception sets is a much coarser point in that space, chosen for its paragraph cost. Java's checked exceptions [8] are the cautionary precedent. ulang avoids their main burden by inferring the raises of private procedures and by having no subtyping among exceptions, so the sets compare exactly.

Uniformity and canonical form. Go [5] made a canonical formatter part of the culture; ulang makes it part of the checker. Unison [21] identifies definitions by the hash of their syntax. ulang uses the e-graph of the prover for the weaker but semantic goal of *detecting* duplicates and simpler equivalents, not of identifying them.

Proving. The simplifier follows the tradition of Isabelle's `simp` [16] and of cooperating decision procedures [3, 15, 19]. The e-graph plan follows egg [22]. Higher-order patterns follow Miller [13], and the structured proof language is in the spirit of Isar and of Lean 4's tactic blocks [4], reduced to a handful of keywords. Enumerated testing follows SmallCheck [18] rather than QuickCheck [2], for determinism.

Languages for agents. We are not aware of a language designed from the premise that its authors learn it in context and its main cost is verification. Most work on agents and code adapts the environment (tools, prompts, retrieval) to existing languages. ulang asks the converse question: what the language should be when the environment is an agent.

12 Conclusion

ulang shows that a language designed around cost to verify and in-context learnability is practical: small enough to explain in a few pages, strict enough that the checker teaches it, and expressive enough to implement its own compiler and prover. Its central choices are pure functions and effectful procedures, modes at every call, atomic effects, checked and inferred exceptions, overloading without preferences, one way to write each construct, a canonical layout, and docs as syntax. Each one removes a question that a reviewer of an agent's code would otherwise have to ask. The plans now in progress make the checker's feedback a protocol with hints and positions on every part, put examples into signatures, test lemmas before proving them, and derive lint from proven equalities. The largest open question is empirical: whether the departures from familiar syntax pay for themselves with agents that have never seen the language. The experiment of Section 10 would answer it, and the design is meant to change according to the result.

Acknowledgement. The implementation of ulang has been developed largely with coding agents, whose trip-ups, recorded in `unclear.md`, informed many of the decisions described here.

Declaration of AI use. This paper was drafted by an AI assistant (Claude Opus 5.5, in Claude Code), working from the repository of ulang: its sources, `CLAUDE.md`, `README.md`, `DESIGN.md`, `TODO.md`, and the plans in progress. The assistant also ran the code examples and error messages quoted here against the current implementation. The design itself, its principles, and the plans the paper describes are the author's, developed together with coding agents. The author is responsible for the content, which remains to be reviewed and revised.

References

[1] John Barnes. *High Integrity Software: The SPARK Approach to Safety and Security*. Addison-Wesley, 2003.

[2] Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *International Conference on Functional Programming (ICFP)*, 2000.

[3] Sylvain Conchon, Evelyne Contejean, Johannes Kanig, and Stéphane Lescuyer. CC(X): Semantic combination of congruence closure with solvable theories. *Electronic Notes in Theoretical Computer Science*, 198(2):51–69, 2008.

[4] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Conference on Automated Deduction (CADE)*, volume 12699 of *LNCS*, 2021.

[5] Alan A. A. Donovan and Brian W. Kernighan. *The Go Programming Language*. Addison-Wesley, 2015.

[6] Gidon Ernst. Loop verification with invariants and contracts. In *Verification, Model Checking, and Abstract Interpretation (VMCAI)*, volume 13182 of *LNCS*. Springer, 2022.

[7] Jean-Christophe Filliâtre and Andrei Paskevich. Why3 — where programs meet provers. In *European Symposium on Programming (ESOP)*, volume 7792 of *LNCS*, 2013.

[8] James Gosling, Bill Joy, and Guy Steele. The Java language specification. Addison-Wesley, 1996.

[9] C. A. R. Hoare. Procedures and parameters: An axiomatic approach. In *Symposium on Semantics of Algorithmic Languages*, volume 188 of *Lecture Notes in Mathematics*. Springer, 1971.

[10] Daan Leijen. Koka: Programming with row polymorphic effect types. In *Mathematically Structured Functional Programming (MSFP)*, 2014.

[11] K. Rustan M. Leino. Dafny: An automatic program verifier for functional correctness. In *Logic for Programming, Artificial Intelligence, and Reasoning (LPAR)*, volume 6355 of *LNCS*, 2010.

[12] Bertrand Meyer. *Object-Oriented Software Construction*. Prentice Hall, 2nd edition, 1997.

[13] Dale Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. *Journal of Logic and Computation*, 1(4):497–536, 1991.

[14] Robin Milner. A theory of type polymorphism in programming. *Journal of Computer and System Sciences*, 17(3):348–375, 1978.

[15] Greg Nelson and Derek C. Oppen. Simplification by cooperating decision procedures. *ACM Transactions on Programming Languages and Systems*, 1(2):245–257, 1979.

[16] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*, volume 2283 of *LNCS*. Springer, 2002.

[17] William Pugh. The Omega test: A fast and practical integer programming algorithm for dependence analysis. In *Supercomputing*, 1991.

[18] Colin Runciman, Matthew Naylor, and Fredrik Lindblad. SmallCheck and Lazy SmallCheck: Automatic exhaustive testing for small values. In *Haskell Symposium*, 2008.

[19] Robert E. Shostak. Deciding combinations of theories. *Journal of the ACM*, 31(1):1–12, 1984.

[20] Thomas Tuerk. Local reasoning about while-loops. In *VSTTE Theory Workshop*, 2010.

[21] Unison Computing. The Unison language. https://www.unison-lang.org.

[22] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. egg: Fast and extensible equality saturation. *Proceedings of the ACM on Programming Languages*, 5(POPL), 2021.